

Composición de Launch para simulación en ROS

Jesús Leonardo Cárdenas Martínez
Unidad Tamaulipas
Cinvestav
Ciudad Victoria, México
jesus.cardenas@cinvestav.mx

Abstract—Este documento describe los patrones de composición de launch files en ROS 2 orientados a simulación multi-robot con Gazebo. Se explica cómo un único launch file orquesta el arranque de un simulador, la descripción de robots, el spawn de modelos, y los nodos de control, coordinando el orden de lanzamiento mediante acciones temporizadas.

Se analiza en detalle la estructura de IncludeLaunchDescription, TimerAction, OpaqueFunction, y el patrón de namespaces para múltiples robots, usando como caso de estudio el launch file de experimentos de consenso de promedio del proyecto de control de formación multi-robot desarrollado en mi investigación para Tesis.

Index Terms—ROS 2, launch files, Gazebo, simulación multi-robot, namespaces, TimerAction

I. Introducción

En un experimento de control de formación multi-robot sobre ROS 2 y Gazebo, el inicio de una simulación involucra docenas de procesos que deben arrancar en un orden específico: primero el simulador, luego las descripciones de cada robot, luego el spawn de cada modelo en el mundo, y finalmente los nodos de control. Coordinar esto manualmente es inviable.

El launch file es el mecanismo de ROS 2 diseñado para resolver exactamente este problema. Es un script Python que describe declarativamente toda la configuración del sistema: qué procesos lanzar, con qué parámetros, en qué secuencia, y bajo qué condiciones. Al ejecutar `ros2 launch`, el sistema interpreta esa descripción y orquesta el arranque completo.

Este documento explica los patrones de composición que aparecen en los launch files de este proyecto, usando `consenso_prom.launch.py` como caso de estudio concreto. El objetivo es que cada línea de ese archivo tenga una justificación clara: por qué existe, qué problema resuelve, y qué pasaría si no estuviera.

II. Estructura básica de un launch file

Un launch file es un módulo Python con una sola función obligatoria: `generate_launch_description()`. Esta función debe retornar un objeto `LaunchDescription`, que no es más que una lista ordenada de acciones. `ros2 launch` importa

el módulo, llama a esa función, y procesa las acciones en secuencia.

A. LaunchDescription y acciones

Una acción es cualquier objeto que el sistema de launch sabe cómo ejecutar. Las acciones más importantes son: `DeclareLaunchArgument` (registra un argumento con valor por defecto), `Node` (lanza un nodo ROS 2), `IncludeLaunchDescription` (ejecuta otro launch file), `OpaqueFunction` (ejecuta una función Python arbitraria), y `TimerAction` (retrasa la ejecución de otras acciones). El Código 1 muestra la forma mínima:

B. DeclareLaunchArgument y LaunchConfiguration

Para que el usuario pueda pasar valores desde la terminal, el launch file los declara con `DeclareLaunchArgument` y los recupera con `LaunchConfiguration`. El valor siempre viaja como string; la conversión de tipos es responsabilidad del usuario.

Cuando el valor se pasa directamente a un nodo sin transformación, `LaunchConfiguration` puede usarse como sustitución en línea: ROS 2 resuelve su valor en el momento de lanzar el proceso sin necesidad de código Python explícito. Cuando se necesita operar sobre el valor (convertir a int, hacer cálculos, iterar), se usa `OpaqueFunction`, que se describe en la sección siguiente.

III. OpaqueFunction: lógica Python en el launch

A. Por qué existe

Las sustituciones en línea de ROS 2 (como `LaunchConfiguration`) son mecanismos declarativos: resuelven un valor diferido en el momento del lanzamiento, pero no pueden ejecutar código Python arbitrario. Si el launch file necesita convertir tipos, hacer cálculos, o generar una lista variable de nodos, ese código no cabe en una sustitución.

`OpaqueFunction` resuelve este límite. Es una acción que, cuando el sistema la procesa, llama a una función Python

Código 1: Estructura mínima de un launch file.

```
1 from launch import LaunchDescription
2 from launch_ros.actions import Node
3
4 def generate_launch_description():
5     return LaunchDescription([
6         Node(
7             package='mi_paquete',
8             executable='mi_nodo',
9             name='mi_nodo',
10            output='screen'
11        )
12    ])
```

y le entrega el `context`¹: el estado global de la ejecución con todos los argumentos ya resueltos. La función retorna una lista de acciones que el sistema agrega a la secuencia de ejecución.

B. El patrón `perform(context)`

Dentro de una `OpaqueFunction`, la forma de leer el valor actual de un argumento es llamar a `.perform(context)` sobre el objeto `LaunchConfiguration` correspondiente. Esto fuerza la resolución inmediata del valor y entrega un string, que el código Python puede operar como necesite. El Código 2 muestra el patrón completo:

Este patrón es el que usa `consenso_prom.launch.py` para generar dinámicamente las acciones de n robots a partir del argumento `n_robots`. Sin `OpaqueFunction`, no sería posible construir un bucle `for` cuya longitud depende de un argumento de la terminal.

C. Firma de la función

La firma `def launch_setup(context, *args, **kwargs)` es obligatoria. El sistema de launch llama a la función pasando el contexto como primer argumento posicional, y puede pasar argumentos adicionales dependiendo de cómo se configure la acción. El uso de `*args, **kwargs` garantiza compatibilidad aunque el sistema agregue parámetros en versiones futuras.

IV. Composición jerárquica con `IncludeLaunchDescription`

A. El problema de la monolítica

Un launch file que lanza Gazebo, describe robots, los spawna, y corre nodos de control podría escribirse como un bloque de código lineal. El problema es que cada uno de esos componentes tiene su propio launch file oficial y bien probado: `gazebo.launch.py` de `gazebo_ros`, `rsp.launch.py`

¹El `context` es el objeto que contiene el estado global de la ejecución del launch: los valores actuales de todos los argumentos ya declarados, las variables de entorno, y otras condiciones del sistema. Es el mecanismo por el que la función tiene acceso a los argumentos resueltos.

de `articubot_one`. Duplicar su contenido es error-prone y rompe la mantenibilidad.

`IncludeLaunchDescription` resuelve esto: permite que un launch file ejecute otro como si fuera una subrutina. El launch file padre construye la secuencia de alto nivel; los hijos manejan sus propios detalles internos.

B. Anatomía de `IncludeLaunchDescription`

La acción recibe dos piezas: la fuente del launch file (cómo localizarlo) y, opcionalmente, un diccionario de argumentos que sobrescribe los valores por defecto del launch hijo. El Código 3 muestra el patrón para incluir Gazebo:

`get_package_share_directory` usa `ament_index` para localizar el directorio de instalación del paquete, independientemente de dónde esté físicamente en el sistema de archivos. Esto hace el código portátil entre distintas instalaciones y workspaces.

C. Pasando argumentos al launch hijo

Cuando el launch hijo declara argumentos propios, el padre puede sobrescribirlos mediante el parámetro `launch_arguments`. El Código 4 muestra cómo se pasa el namespace al launch de descripción del robot:

El valor de `ns` es un string Python calculado dentro de `launch_setup`. Notar que `use_sim_time` se pasa como string `'false'`: los argumentos de launch siempre son strings, sin excepción.

D. `spawn_entity.py` como nodo auxiliar

El spawn de un modelo en Gazebo no se hace mediante un launch file sino mediante un nodo especial de `gazebo_ros` que se ejecuta una sola vez y termina. Se lanza con la acción `Node` normal, pero sus argumentos de línea de comandos (posición, nombre de entidad, fuente del modelo) se pasan mediante el parámetro `arguments`, no mediante `parameters`. El Código 5 muestra el patrón:

El flag `-topic` indica que `spawn_entity.py` debe leer la descripción del robot desde el tópico `/robot0/robot_description`, que es el que publica el

Código 2: Patrón OpaqueFunction con perform(context).

```
1 from launch.actions import OpaqueFunction
2 from launch.substitutions import LaunchConfiguration
3
4 def launch_setup(context, *args, **kwargs):
5     # .perform(context) extrae el string del argumento
6     n = int(LaunchConfiguration('n_robots').perform(context))
7     R = float(LaunchConfiguration('neighbor_radius').perform(context))
8
9     actions = []
10    for i in range(n):
11        actions.append(
12            Node(
13                package='control_nodes',
14                executable='consenso_node',
15                namespace=f'robot{i}',
16                parameters=[{'neighbor_radius': R}],
17            )
18        )
19    return actions
20
21 def generate_launch_description():
22    return LaunchDescription([
23        DeclareLaunchArgument('n_robots', default_value='3'),
24        DeclareLaunchArgument('neighbor_radius', default_value='5.0'),
25        OpaqueFunction(function=launch_setup),
26    ])
```

nodo RSP (Robot State Publisher²) lanzado instantes antes. Por eso el spawn se retrasa: necesita que el RSP esté publicando antes de intentar leer el tópico.

V. Coordinación temporal con TimerAction

A. Por qué hay que escalonar el arranque

Gazebo necesita tiempo para inicializarse antes de que otro proceso pueda comunicarse con él. Si el nodo de spawn se lanza inmediatamente después de Gazebo, intentará conectarse a un simulador que aún está cargando y fallará. El mismo problema ocurre con el RSP: el spawn necesita leer el tópico `robot_description`, que el RSP debe estar publicando. Y el nodo de control no debe arrancar hasta que el robot esté spawnado y con odometría disponible.

Esta cadena de dependencias temporales se resuelve con `TimerAction`: una acción que retrasa la ejecución de un conjunto de acciones hijas un número fijo de segundos desde que el launch file comenzó.

B. Anatomía de TimerAction

`TimerAction` recibe dos parámetros: `period` (el retardo en segundos, como float) y `actions` (una lista de acciones a ejecutar cuando el timer dispare). El Código 6 muestra la forma básica:

²El Robot State Publisher (RSP) es el nodo estándar de ROS 2 que lee un archivo URDF y publica la descripción del robot como string en el tópico `robot_description`, además de calcular y publicar las transformadas entre los eslabones del robot en el tópico `/tf`.

El timer comienza a contar desde el momento en que el launch file empieza a ejecutarse, no desde que la acción anterior termina. Esto significa que todos los timers se lanzan en paralelo al inicio, y cada uno dispara de forma independiente cuando su period se cumple. No hay dependencia explícita entre timers.

C. El esquema de retardos en `consenso_prom`

En `consenso_prom.launch.py` se usa un esquema de retardos calculados para escalonar el spawn de n robots sin sobrecargar Gazebo. El Código 7 muestra la lógica:

La secuencia para cada robot i es: RSP en delay, spawn en delay + 0.5 s, nodo de control en delay + 1.5 s. Los 0.5 s entre RSP y spawn dan tiempo para que el tópico `robot_description` esté disponible. El segundo extra antes del nodo de control da tiempo para que el robot aparezca en la simulación y empiece a publicar odometría.

Al separar robots con 1.5 s entre sí, los 8 robots completan su secuencia de arranque en unos 13.5 s, sin que ningún paso de spawn compita con otro por los recursos de Gazebo.

D. Los nodos tardíos

Los nodos que necesitan que todos los robots estén listos (el grafo de conectividad AIRE³ y los plotters) se lanzan

³AIRE es el nodo que calcula la conectividad entre robots basada en distancia euclidiana. Necesita que todos los robots estén publicando odometría antes de arrancar, de lo contrario podría reportar grafos incompletos durante los primeros ciclos.

Código 3: Inclusión de gazebo.launch.py desde el launch padre.

```
1 from launch.actions import IncludeLaunchDescription
2 from launch.launch_description_sources import (
3     PythonLaunchDescriptionSource
4 )
5 from ament_index_python.packages import get_package_share_directory
6 import os
7
8 gazebo_launch = os.path.join(
9     get_package_share_directory('gazebo_ros'),
10    'launch',
11    'gazebo.launch.py'
12 )
13
14 actions.append(
15     IncludeLaunchDescription(
16         PythonLaunchDescriptionSource(gazebo_launch)
17     )
18 )
```

Código 4: Inclusión de rsp.launch.py con argumentos.

```
1 rsp_node = IncludeLaunchDescription(
2     PythonLaunchDescriptionSource(rsp_launch),
3     launch_arguments={
4         'namespace': ns,          # ej: 'robot0'
5         'use_sim_time': 'false'
6     }.items()
7 )
```

con un retardo calculado a partir del último robot:

El margen de 2.0 s adicional después del último robot garantiza que los nodos tardíos no arranquen antes de que el último robot de control esté operativo.

VI. El patrón namespace para sistemas multi-robot

A. El problema del nombre único

Cuando dos robots ejecutan el mismo nodo de control, ambos intentan publicar en /cmd_vel y suscribirse a /odom. DDS conecta a todos los publicadores y suscriptores del mismo tópico entre sí, sin distinción. Robot 0 recibiría la odometría de robot 1 y viceversa; los comandos de velocidad se mezclarían entre ambos. El sistema colapsaría.

La solución es el namespace: un prefijo jerárquico que califica todos los nombres de tópicos de un nodo. Un nodo con namespace robot0 que publica en cmd_vel en realidad publica en /robot0/cmd_vel. Un nodo con namespace robot1 publica en /robot1/cmd_vel. Los dos canales son distintos y no interfieren.

B. Asignación de namespace en el launch file

El namespace se asigna mediante el parámetro namespace de la acción Node. Todos los tópicos, servicios, parámetros y el propio nombre del nodo quedan bajo ese prefijo automáticamente, sin necesidad de modificar el código del nodo. El Código 9 muestra el patrón:

Con namespace='robot0' y name='consenso', el nombre completo del nodo en el grafo ROS 2 es /robot0/consenso. Sus tópicos quedan en /robot0/cmd_vel, /robot0/odom, etc.

C. Namespace en IncludeLaunchDescription

El mismo principio aplica cuando se incluye el launch file de descripción del robot. El namespace debe pasarse como argumento al launch hijo para que el RSP publique el tópico robot_description bajo el namespace correcto:

Internamente, rsp.launch.py usa ese argumento para asignar el namespace al nodo robot_state_publisher, que entonces publica en /robot0/robot_description. El nodo de spawn lee ese tópico exacto mediante el flag -topic /ns/robot_description, cerrando la cadena.

D. Verificación desde la terminal

Con un sistema multi-robot corriendo, la CLI de ROS 2 permite inspeccionar la separación por namespaces desde

Código 5: Spawn de un modelo de robot en Gazebo.

```

1 spawn_node = Node(
2     package='gazebo_ros',
3     executable='spawn_entity.py',
4     arguments=[
5         '-topic', f'/{ns}/robot_description',
6         '-entity', ns,
7         '-x', str(round(x0, 4)),
8         '-y', str(round(y0, 4)),
9         '-z', '0.1',
10        '-Y', str(round(yaw, 4)),
11    ],
12    output='screen'
13 )

```

Código 6: Estructura básica de TimerAction.

```

1 from launch.actions import TimerAction
2
3 actions.append(
4     TimerAction(period=3.0, actions=[
5         Node(
6             package='gazebo_ros',
7             executable='spawn_entity.py',
8             ...
9         )
10    ])
11 )

```

Código 7: Esquema de retardos escalonados para n robots.

```

1 for i, (x0, y0, yaw) in enumerate(poses):
2     ns = f'robot{i}'
3     delay = 3.0 + i * 1.5
4     # robot0: 3.0 s, robot1: 4.5 s, ..., robot7: 13.5 s
5
6     actions.append(TimerAction(period=delay,
7                                actions=[rsp_node]))
8     actions.append(TimerAction(period=delay + 0.5,
9                                actions=[spawn_node]))
10    actions.append(TimerAction(period=delay + 1.5,
11                               actions=[control_node]))

```

la Terminal 11:

VII. Análisis de consenso_prom.launch.py

A. Visión general

consenso_prom.launch.py es el launch file principal del experimento de consenso de promedio. Orquesta el arranque de Gazebo, n robots (hasta 8), el nodo de conectividad AIRE, y dos nodos de graficación. Todo cabe en un único archivo gracias a la composición de los patrones descritos en las secciones anteriores.

La estructura del archivo sigue el patrón OpaqueFunction: generate_launch_description declara los argumentos y delega toda la lógica de construcción a launch_setup. Esto es necesario porque el número de nodos a lanzar depende del argumento n_robots, que no se conoce hasta tiempo de ejecución.

B. Argumentos declarados

El Código 12 muestra los seis argumentos que el usuario puede sobrescribir desde la terminal:

Código 8: Retardo para nodos que dependen de todos los robots.

```
1 late_delay = 3.0 + n * 1.5 + 2.0
2 # Para n=8: 3.0 + 12.0 + 2.0 = 17.0 s
3
4 actions.append(TimerAction(period=late_delay, actions=[
5     Node(package='simulador', executable='aire', ...)
6 ]))
```

Código 9: Asignación de namespace a un nodo desde el launch file.

```
1 for i in range(n):
2     ns = f'robot{i}'    # 'robot0', 'robot1', ...
3
4     Node(
5         package='control_nodes',
6         executable='consenso_node',
7         name='consenso',
8         namespace=ns,          # <- aqui
9         parameters=[{...}],
10        output='screen'
11    )
```

Código 10: Namespace propagado al launch de descripción del robot.

```
1 rsp_node = IncludeLaunchDescription(
2     PythonLaunchDescriptionSource(rsp_launch),
3     launch_arguments={
4         'namespace': ns,
5         'use_sim_time': 'false'
6     }.items()
7 )
```

Un experimento típico con 4 robots y 30 segundos se lanza con el comando de la Terminal 13:

C. Poses iniciales predefinidas

Las posiciones y orientaciones iniciales de los 8 robots están hardcodeadas como una constante `POSES_8` al inicio del archivo. El diseño del layout garantiza conectividad con radio $R = 5.0$ m: ningún par de robots está a más de 5 m de distancia de todos sus vecinos. Al usar solo `n_robots` de los 8 disponibles se toman los primeros `n` de la lista, lo que permite ejecutar experimentos con subconjuntos del layout completo sin rediseñar las poses.

D. Secuencia de lanzamiento

La función `launch_setup` construye la lista `actions` en el siguiente orden:

1. **Gazebo.** La primera acción (sin timer) es la inclusión de `gazebo.launch.py`. Arranca inmediatamente, dando el mayor tiempo posible para que el simulador se inicialice antes de que lleguen los primeros spawns.

2. **Obstáculo opcional.** Si `spawn_obstacle = true`, se agrega un `TimerAction` a 3.0 s que spawna un cilindro en la posición indicada. El retardo de 3 s da tiempo a Gazebo para estar listo.

3. **Robots (bucle).** Para cada robot i , con retardo base $\text{delay} = 3.0 + i * 1.5$:

- RSP en delay s.
- Spawn en delay + 0.5 s.
- Nodo de control en delay + 1.5 s.

4. **Nodos tardíos.** Una vez que el último robot está operativo ($\text{late_delay} = 3.0 + n * 1.5 + 2.0$), se lanzan AIRE y los dos plotters.

El Diagrama 14 ilustra la línea de tiempo para `n_robots=3`:

E. Parámetros del nodo de control

El nodo `consenso_node` recibe sus parámetros directamente en el launch file, sin archivo YAML. Esto es práctico para experimentos: los valores pueden cambiarse sin recompilar ni mantener archivos de configuración adi-

Terminal 11: Inspección de nodos y tópicos con namespaces.

```
1 # Lista todos los nodos activos
2 ros2 node list
3 # /robot0/consenso
4 # /robot1/consenso
5 # ...
6
7 # Lista topicos de un robot especifico
8 ros2 topic list | grep robot0
9 # /robot0/cmd_vel
10 # /robot0/odom
11 # /robot0/scan
```

Código 12: Argumentos de consenso_prom.launch.py.

```
1 DeclareLaunchArgument('n_robots',      default_value='8'),
2 DeclareLaunchArgument('neighbor_radius', default_value='5.0'),
3 DeclareLaunchArgument('t_final',        default_value='60.0'),
4 DeclareLaunchArgument('spawn_obstacle', default_value='false'),
5 DeclareLaunchArgument('obstacle_x',     default_value='4.0'),
6 DeclareLaunchArgument('obstacle_y',     default_value='4.5'),
```

Terminal 13: Lanzamiento del experimento con argumentos personalizados.

```
1 ros2 launch simulador consenso_prom.launch.py n_robots:=4 t_final:=30.0
```

Diagrama 14: Línea de tiempo de arranque para n_robots=3.

```
1 t=0.0 s -> Gazebo arranca
2 t=3.0 s -> RSP robot0 | spawn_obstacle (si aplica)
3 t=3.5 s -> spawn robot0
4 t=4.5 s -> control robot0 | RSP robot1
5 t=5.0 s -> spawn robot1
6 t=6.0 s -> control robot1 | RSP robot2
7 t=6.5 s -> spawn robot2
8 t=7.5 s -> control robot2
9 t=9.5 s -> AIRE + plotters (3.0 + 3*1.5 + 2.0)
```

cionales. El Código 15 muestra los parámetros configurados:

VIII. Conclusiones

Un launch file de simulación multi-robot combina cuatro patrones que trabajan en conjunto:

OpaqueFunction permite que el número de nodos a lanzar sea variable y dependa de argumentos de la terminal. Sin él, el launch file solo puede declarar nodos de forma estática.

IncludeLaunchDescription permite reutilizar los launch files oficiales de Gazebo y RSP sin duplicar su

contenido. El launch padre construye la secuencia de alto nivel; los hijos se encargan de sus detalles.

TimerAction resuelve las dependencias temporales entre componentes. Gazebo necesita tiempo antes de recibir spawns; el RSP necesita tiempo antes de que el spawn lea su tópico; el nodo de control necesita que el robot exista antes de intentar leer odometría.

Namespaces permiten que múltiples instancias del mismo nodo coexistan sin interferencia. Es el único mecanismo para separar los canales de comunicación de distintos robots sin modificar el código del nodo.

Los Scripts 16–18 muestran el launch file completo del experimento dividido en sus tres bloques principales:

Código 15: Parámetros del nodo de consenso en el launch file.

```
1 control_node = Node(
2     package='control_nodes',
3     executable='consenso_node',
4     name='consenso',
5     namespace=ns,
6     parameters=[{
7         'consensus_type': 'prom_err',
8         'k1': 0.8,
9         'k2': 1.0,
10        'vmax': 1.0,
11        'wmax': 1.0,
12    }],
13     output='screen'
14 )
```

Script 16: consenso_prom.launch.py – Parte 1: imports y poses iniciales.

```
1 import os
2 import math
3
4 from launch import LaunchDescription
5 from launch.actions import (
6     DeclareLaunchArgument, IncludeLaunchDescription,
7     OpaqueFunction, TimerAction
8 )
9 from launch.launch_description_sources import PythonLaunchDescriptionSource
10 from launch.substitutions import LaunchConfiguration
11 from launch_ros.actions import Node
12 from ament_index_python.packages import get_package_share_directory
13
14 # Posiciones (x, y, yaw) de los 8 robots en el mundo de Gazebo.
15 # El layout esta disenado para que todos los pares esten a menos
16 # de 5.0 m entre si, garantizando conectividad con ese radio.
17 POSES_8 = [
18     (0.0, 0.0, math.pi / 4),      # robot0 -- esquina BL
19     (4.0, 1.0, -math.pi / 2),     # robot1 -- borde inferior
20     (8.0, 0.0, -math.pi / 4),     # robot2 -- esquina BR
21     (1.0, 4.0, math.pi),          # robot3 -- borde izquierdo
22     (8.0, 4.0, 0.0),              # robot4 -- borde derecho
23     (0.0, 8.0, 3*math.pi / 4),     # robot5 -- esquina TL
24     (4.0, 9.0, math.pi / 2),       # robot6 -- borde superior
25     (8.0, 8.0, -3*math.pi / 4),    # robot7 -- esquina TR
26 ]
```


Script 17: consenso_prom.launch.py – Parte 2: construccion dinamica de acciones.

```
1 def launch_setup(context, *args, **kwargs):
2
3     # Leer y convertir los argumentos ya resueltos por el contexto
4     n = int(LaunchConfiguration('n_robots').perform(context))
5     R = float(LaunchConfiguration('neighbor_radius').perform(context))
6     t_final = float(LaunchConfiguration('t_final').perform(context))
7     spawn_obstacle = LaunchConfiguration('spawn_obstacle').perform(context).lower() == '
8         true'
9     obs_x = float(LaunchConfiguration('obstacle_x').perform(context))
10    obs_y = float(LaunchConfiguration('obstacle_y').perform(context))
11
12    poses = POSES_8[:n]
13    simulador_pkg = get_package_share_directory('simulador')
14    obstacle_sdf = os.path.join(simulador_pkg, 'models', 'obstacle_cylinder.sdf')
15    articubot_pkg = get_package_share_directory('articubot_one')
16    gazebo_launch = os.path.join(get_package_share_directory('gazebo_ros'),
17        'launch', 'gazebo.launch.py')
18    rsp_launch = os.path.join(articubot_pkg, 'launch', 'rsp.launch.py')
19
20    actions = []
21
22    # 1. Gazebo arranca de inmediato para tener el mayor tiempo de
23    # inicializacion antes de que lleguen los primeros spawns.
24    actions.append(
25        IncludeLaunchDescription(PythonLaunchDescriptionSource(gazebo_launch))
26    )
27
28    # 2. Obstaculo opcional: se spawna a los 3 s si el argumento lo pide.
29    if spawn_obstacle:
30        actions.append(TimerAction(period=3.0, actions=[
31            Node(package='gazebo_ros', executable='spawn_entity.py',
32                arguments=['-file', obstacle_sdf, '-entity', 'obstacle_cylinder',
33                    '-x', str(obs_x), '-y', str(obs_y), '-z', '0.5'],
34                output='screen')
35        ]))
36
37    # 3. Por cada robot: RSP -> spawn -> control, con 1.5 s entre robots
38    # para no saturar Gazebo durante el arranque masivo.
39    for i, (x0, y0, yaw) in enumerate(poses):
40        ns = f'robot{i}'
41        delay = 3.0 + i * 1.5
42
43        rsp_node = IncludeLaunchDescription(
44            PythonLaunchDescriptionSource(rsp_launch),
45            launch_arguments={'namespace': ns, 'use_sim_time': 'false'}.items()
46        )
47        spawn_node = Node(
48            package='gazebo_ros', executable='spawn_entity.py',
49            arguments=['-topic', f'/{ns}/robot_description', '-entity', ns,
50                '-x', str(round(x0,4)), '-y', str(round(y0,4)),
51                '-z', '0.1', '-Y', str(round(yaw,4))],
52            output='screen'
53        )
```

Script 18: consenso_prom.launch.py – Parte 3: timers, nodos tardios y punto de entrada.

```
1 control_node = Node(
2     package='control_nodes', executable='consenso_node',
3     name='consenso', namespace=ns,
4     parameters=[{'consensus_type': 'prom_err',
5                  'k1': 0.8, 'k2': 1.0, 'vmax': 1.0, 'wmax': 1.0}],
6     output='screen'
7 )
8 actions.append(TimerAction(period=delay, actions=[rsp_node]))
9 actions.append(TimerAction(period=delay + 0.5, actions=[spawn_node]))
10 actions.append(TimerAction(period=delay + 1.5, actions=[control_node]))
11
12 # 4. Nodos de analisis: esperan a que el ultimo robot este operativo.
13 late_delay = 3.0 + n * 1.5 + 2.0
14 actions.append(TimerAction(period=late_delay, actions=[
15     Node(package='simulador', executable='aire', name='aire',
16         parameters=[{'neighbor_radius': R}], output='screen')
17 ]))
18 actions.append(TimerAction(period=late_delay, actions=[
19     Node(package='simulador', executable='states_plotter_v2',
20         parameters=[{'save_dir': 'figuras/consenso_prom',
21                     't_final': t_final, 'n_robots_expected': n}],
22         output='screen')
23 ]))
24 actions.append(TimerAction(period=late_delay, actions=[
25     Node(package='simulador', executable='pva_plotter',
26         parameters=[{'save_dir': 'figuras/consenso_prom', 't_final': t_final}],
27         output='screen')
28 ]))
29 return actions
30
31
32 # Punto de entrada: declara los argumentos configurables
33 # y delega la construccion a launch_setup via OpaqueFunction.
34 def generate_launch_description():
35     return LaunchDescription([
36         DeclareLaunchArgument('n_robots', default_value='8'),
37         DeclareLaunchArgument('neighbor_radius', default_value='5.0'),
38         DeclareLaunchArgument('t_final', default_value='60.0'),
39         DeclareLaunchArgument('spawn_obstacle', default_value='false'),
40         DeclareLaunchArgument('obstacle_x', default_value='4.0'),
41         DeclareLaunchArgument('obstacle_y', default_value='4.5'),
42         OpaqueFunction(function=launch_setup)
43     ])
```